INFORME DE ACTIVIDAD PRÁCTICA

Sesión 9 · Registro de ventas cabecera–detalle y consulta de ventas

Lenguaje de Programación II · Unidad II · Reto 01

Estudiante	**Yeremin Ccapa Chacca**
Docente	Reyna Barreto Benjamin David
Ciclo / semestre	IV · 2026-2
Fecha de sesión	8 de octubre de 2026
Proyecto	PharmaSoft (Angular 22) + PharmaBackend (Spring Boot 4)

1. Arquitectura y estructura implementada

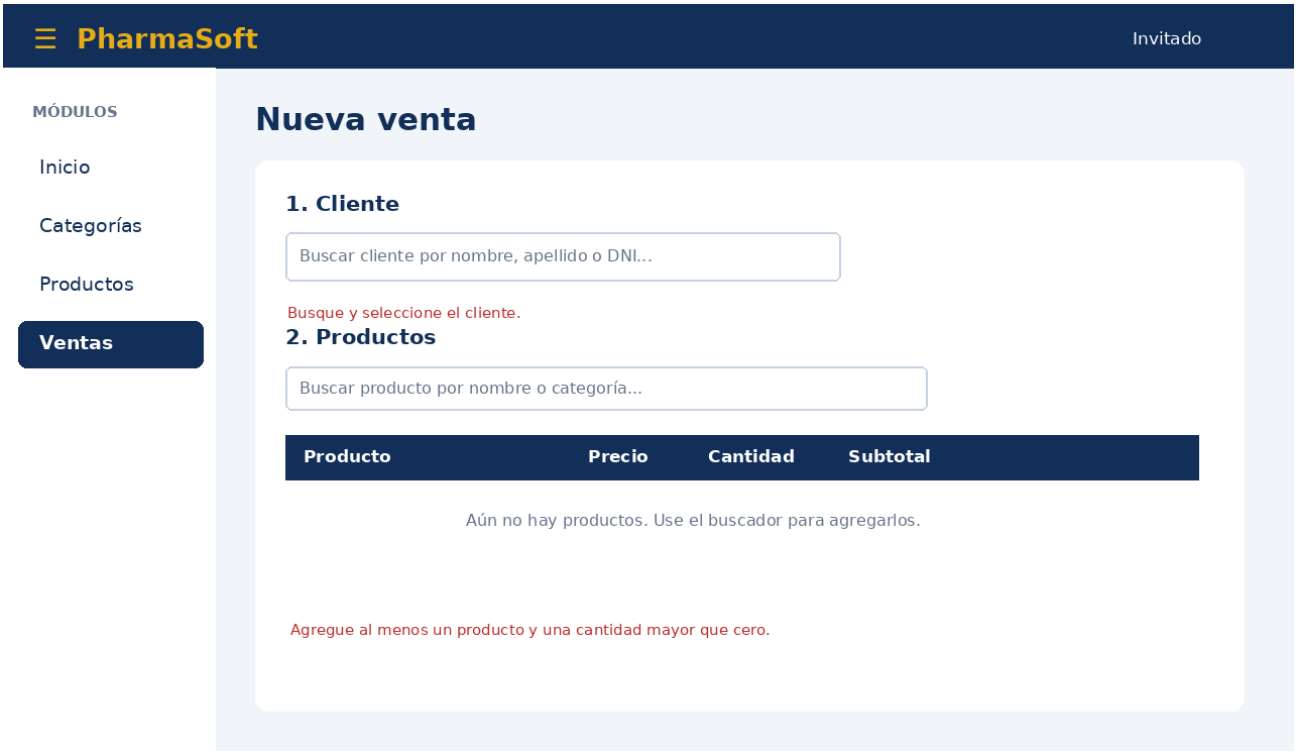
En el proyecto adjunto se identifican las rutas /ventas, /ventas/nueva y /ventas/:id; los componentes VentaList, VentaForm y VentaDetalle; el modelo de venta y el servicio VentaService. El servicio utiliza POST /ventas, GET /ventas/{id} y GET /ventas/buscar respecto de environment.apiUrl.

Elemento	Responsabilidad
venta-form.ts / .html	Formulario reactivo, FormArray, buscadores, validación, cálculos y confirmación.
venta-list.ts / .html	Filtros, paginación, consulta y vista imprimible.
venta-detalle.ts / .html	Obtención de venta, visualización e impresión de comprobante.
venta-service.ts	Comunicación HTTP con el backend; filtros enviados según valor.

2. Desarrollo funcional documentado

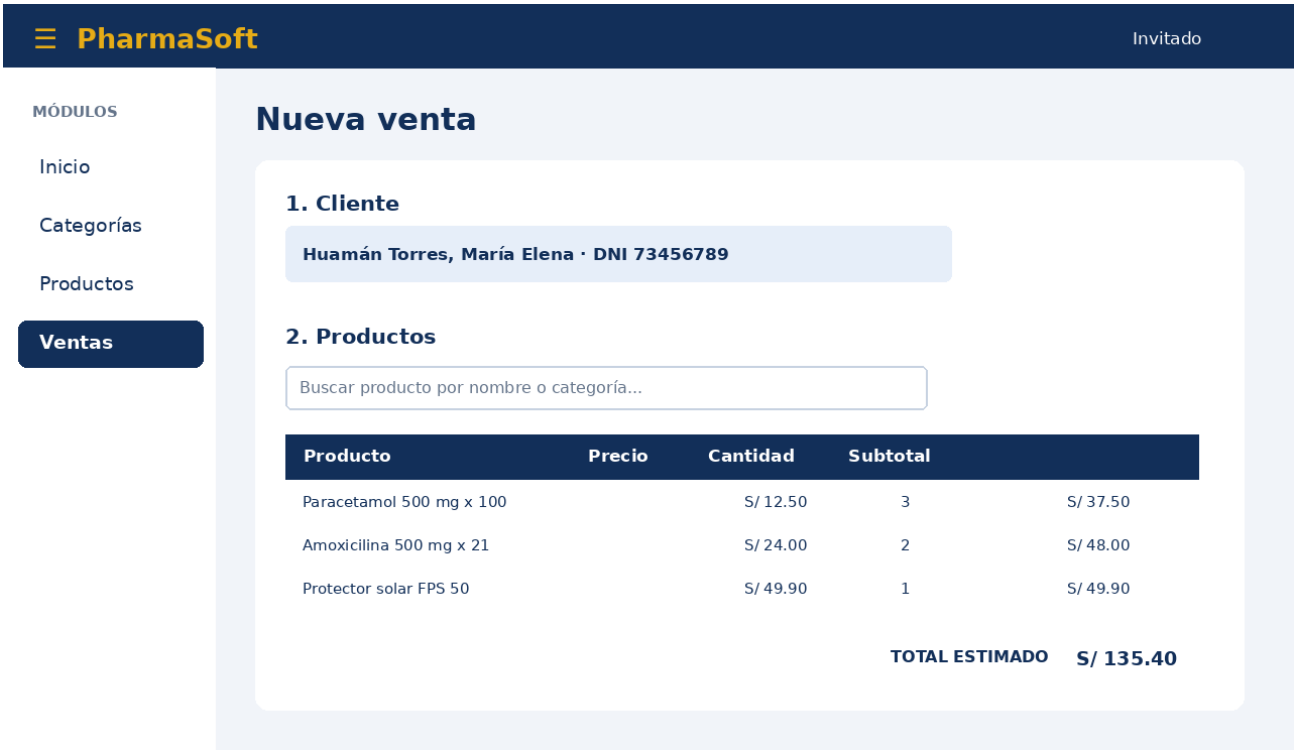
2.1. Cabecera y detalle dinámico

El formulario cuenta con clienteId obligatorio y detalles como FormArray. Cada fila tiene productoId y cantidad. Los buscadores usan FormControl independientes y la coincidencia de texto se calcula mediante señales. Agregar un producto repetido incrementa la cantidad de la línea existente.



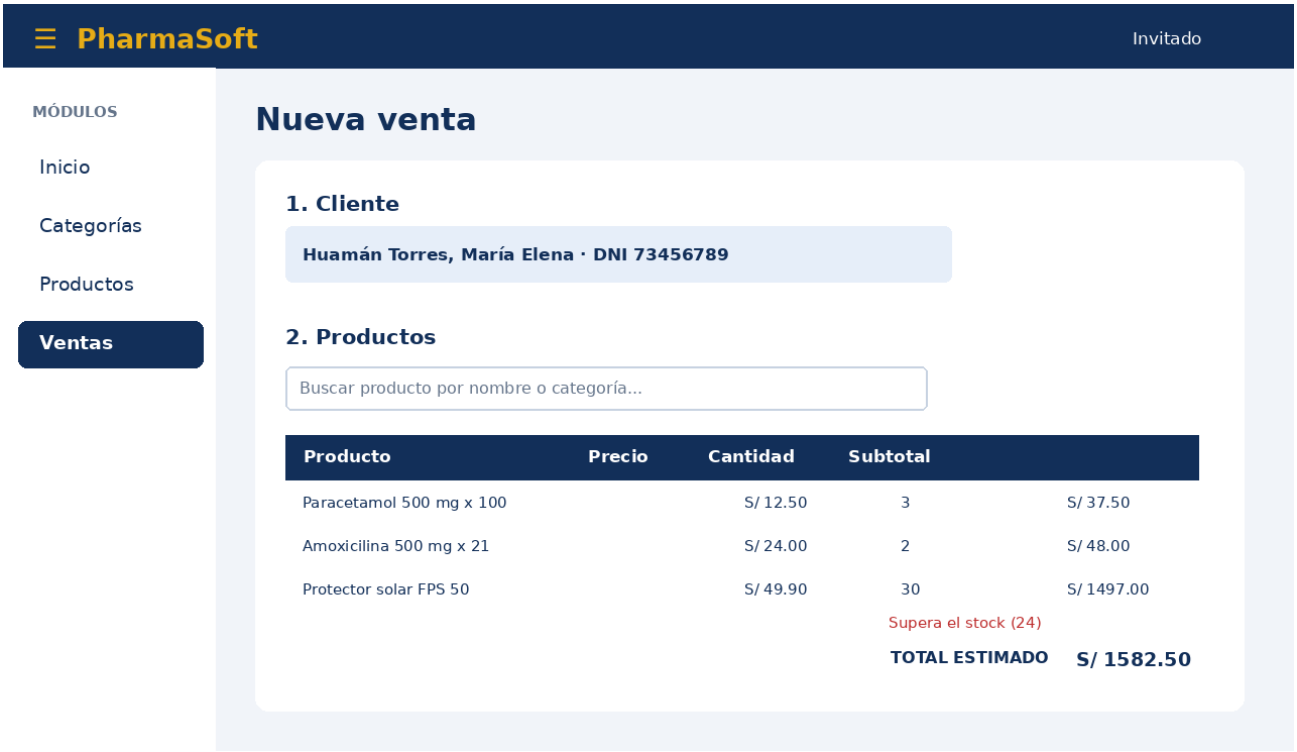
2.2. Cálculos automáticos

El componente convierte valueChanges a signal con toSignal; computed calcula precio, subtotal redondeado a dos decimales y total estimado. El servidor es quien determina los precios y el total definitivo, por lo que el request no debe enviar montos.



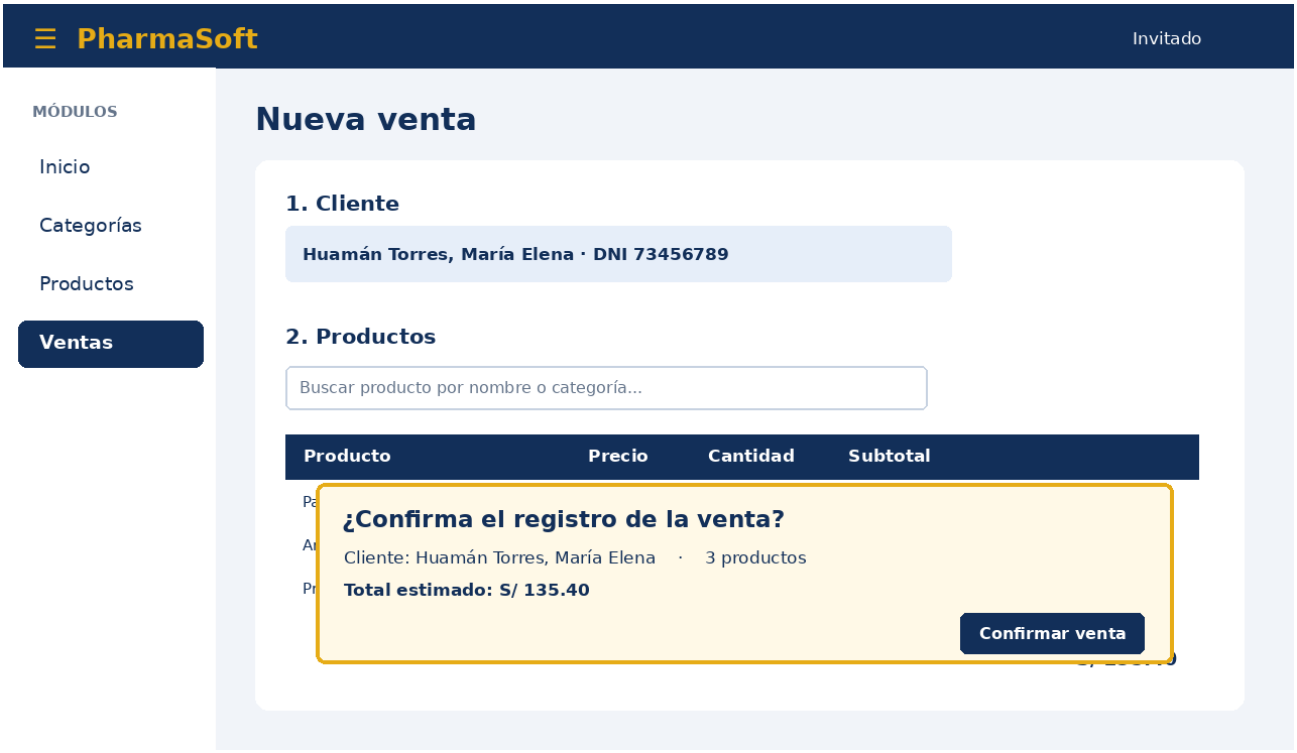
2.3. Validaciones y stock

Se exige cliente, al menos un detalle y cantidades enteras mayores a cero. Una cantidad superior al stock provoca un aviso por línea y evita avanzar al paso de revisión. Los conflictos del backend deben presentarse al usuario, sin registrar parcialmente la operación.



2.4. Revisión y confirmación

revisar() valida el formulario y abre un resumen. confirmar() construye VentaRequest con clienteId y líneas de productoId/cantidad y efectúa la petición POST. Ante un error, la confirmación se cierra y se muestra el mensaje recibido.



2.5. Comprobante imprimible

VentaDetalle obtiene la venta por identificador y consulta los datos del cliente para mostrar su DNI si la consulta está disponible. Incluye un botón que llama a window.print(), además de la apertura automática mediante el parámetro imprimir=1.



Figura. Comprobante recreado a partir del flujo de la guía (datos ficticios). Imagen recreada con datos de ejemplo; no constituye captura de ejecución real.

2.6. Consulta de ventas

La consulta usa filtros opcionales clienteId, estado, desde y hasta y controla que desde no sea posterior a hasta. Emplea navegación paginada, acceso al comprobante y opción de impresión de la consulta.



3. Contrato REST y reglas transaccionales

Operación	Ruta	Validación / respuesta esperada
Registrar	POST /api/v1/ventas	201 si se registra; 400, 404 o 409 según validación de API.
Obtener	GET /api/v1/ventas/{id}	200 o 404.
Buscar	GET /api/v1/ventas/buscar	200 paginado; 409 para fechas inválidas según la guía.

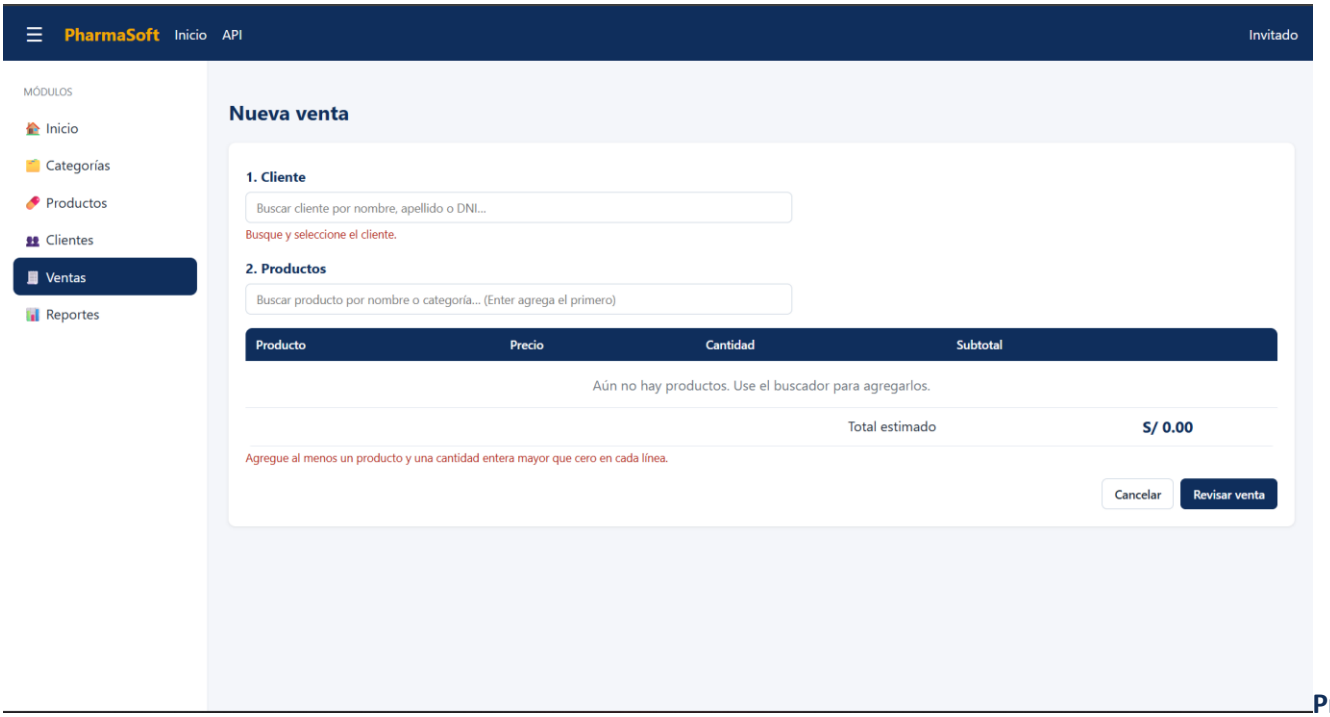
Según la guía, el backend realiza el descuento de stock y la persistencia de cabecera y detalle en una única transacción. No se adjuntó el backend ni resultados de red; esta propiedad se documenta como contrato esperado, no como comprobación experimental.

. Matriz de pruebas funcionales de la guía

Estado “Por validar” significa que el comportamiento se identifica como esperado en la guía pero no se dispone de una captura auténtica o bitácora de ejecución que confirme el resultado. Reemplazar con evidencia propia al ejecutar las pruebas.

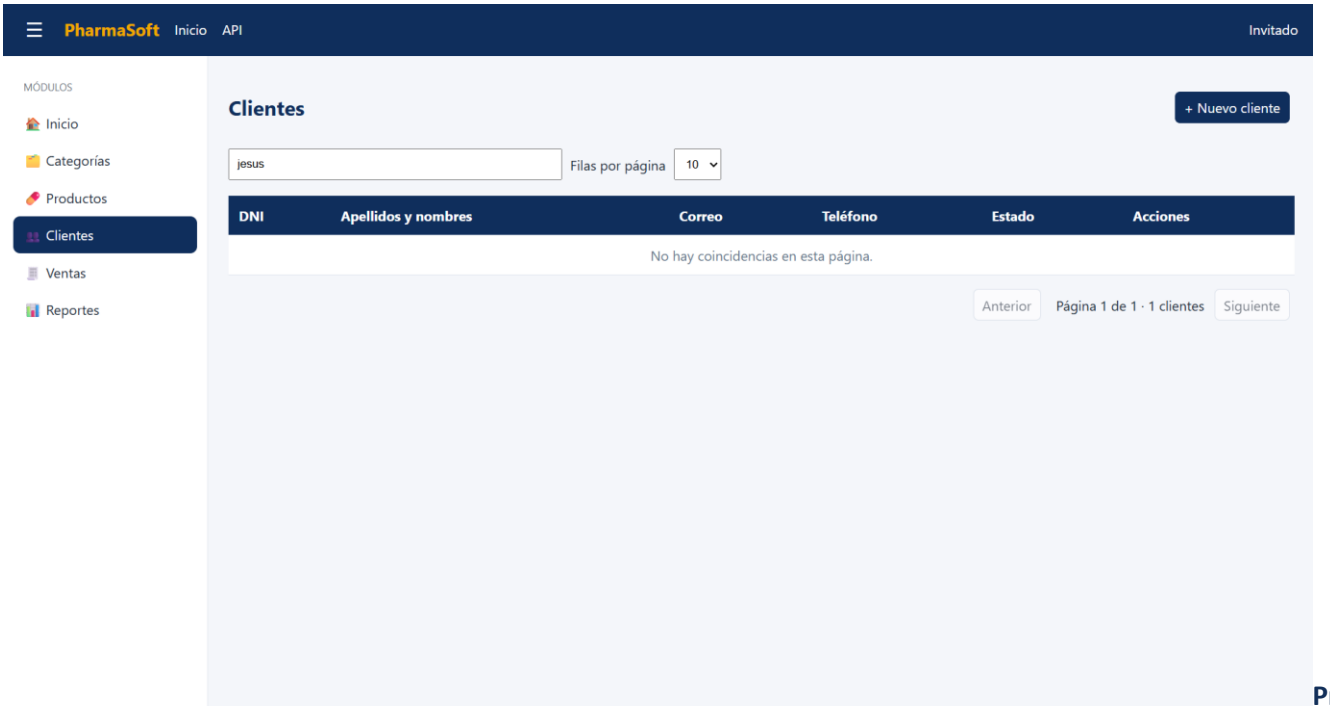
Prueba 01. Formulario vacío

Resultado obtenido: El sistema mostró correctamente los mensajes de validación correspondientes al cliente obligatorio y al detalle de productos vacío. Asimismo, el total estimado permaneció en S/ 0.00 y no se abrió el panel de confirmación.



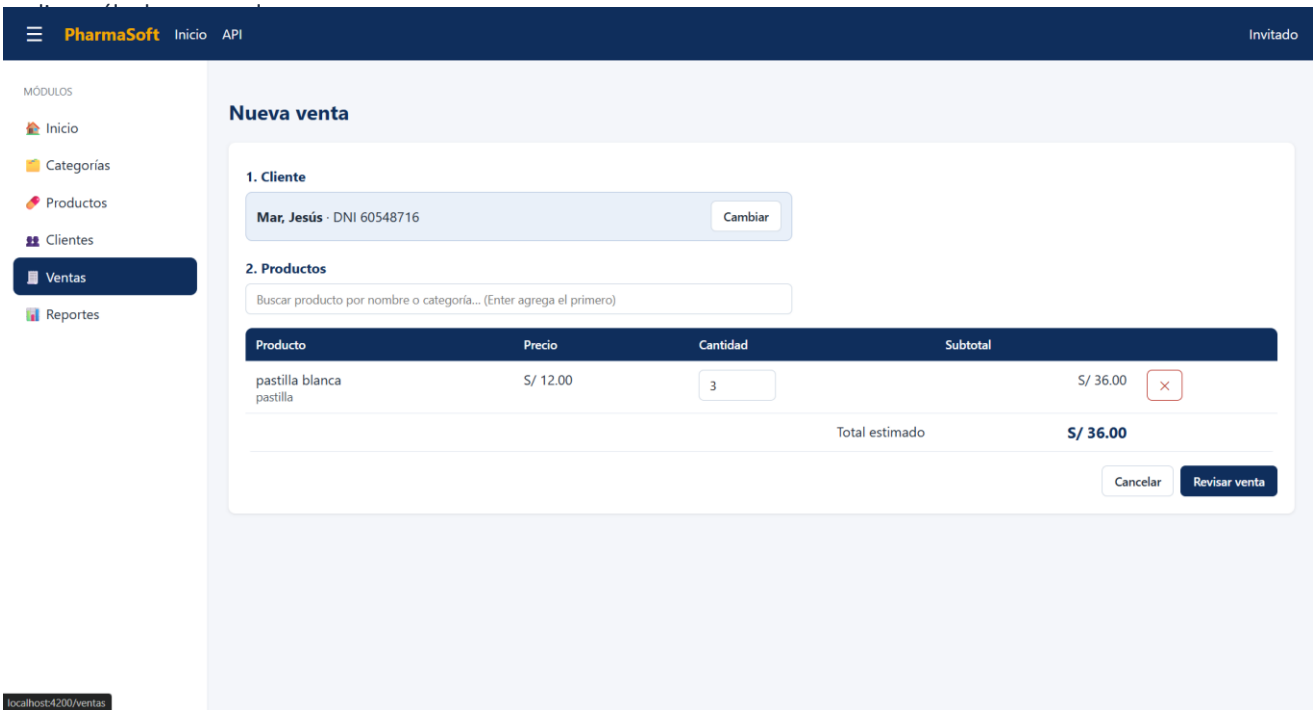
Prueba 02. Cliente sin tilde / inexistente

Resultado obtenido: El sistema realizó la búsqueda y mostró el mensaje «No hay coincidencias en esta página», debido a que ningún cliente visible coincidía con el criterio ingresado. La aplicación permaneció estable y no presentó errores durante la consulta.

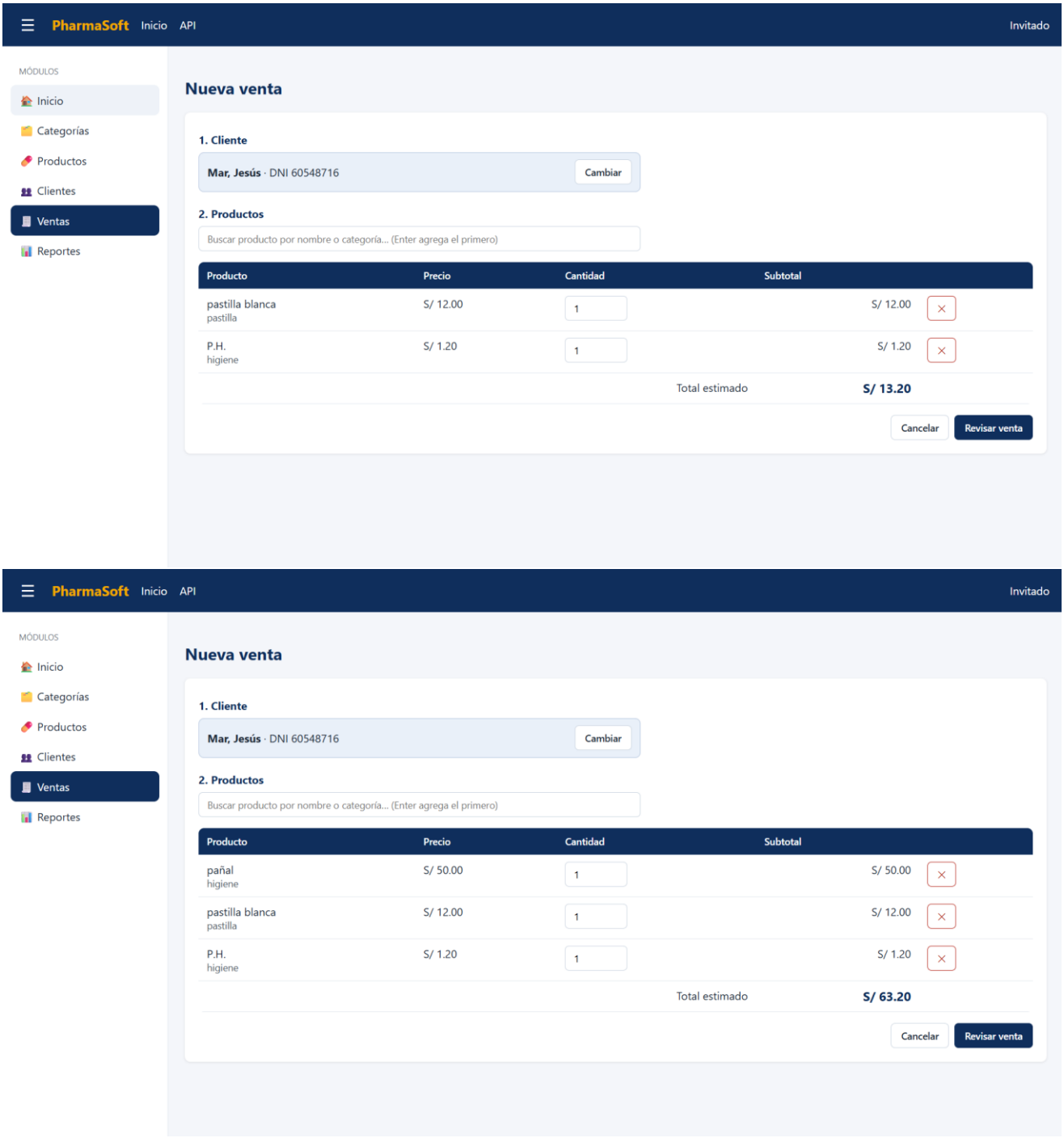


Prueba 03. Agregar producto y cantidad 3

Resultado obtenido: El sistema calculó automáticamente un subtotal de S/ 36.00, correspondiente a la multiplicación del precio unitario por la cantidad ingresada (S/ 12.00 × 3). Asimismo, el total estimado se actualizó correctamente a S/ 36.00, sin necesidad de



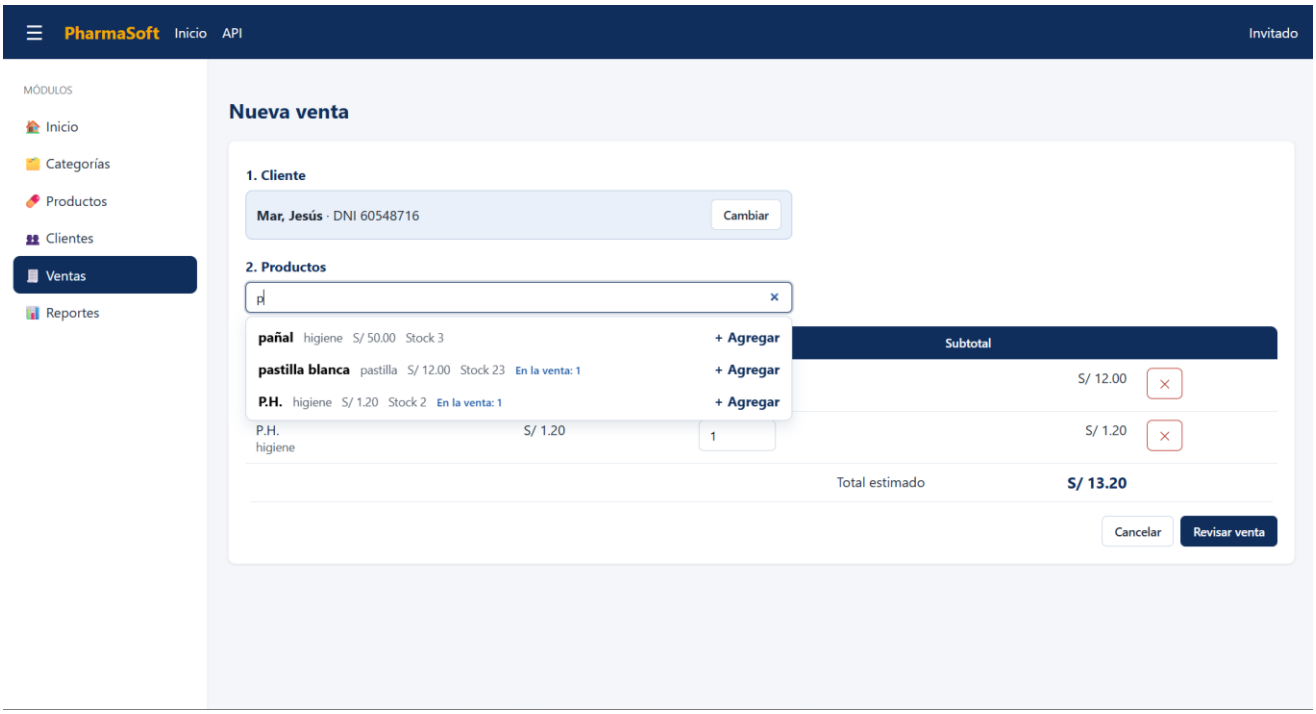
Prueba 04. Enter, agregar y quitar Resultado obtenido: El sistema permitió agregar correctamente dos productos adicionales mediante la tecla Enter, incorporándolos automáticamente al detalle de la venta sin enviar el formulario. Posteriormente, al eliminar uno de los productos mediante el botón X, la fila correspondiente desapareció de la tabla y el total estimado se recalculó automáticamente, descontando el subtotal del producto eliminado. De esta manera, se comprobó el funcionamiento del detalle dinámico y la actualización de los cálculos.

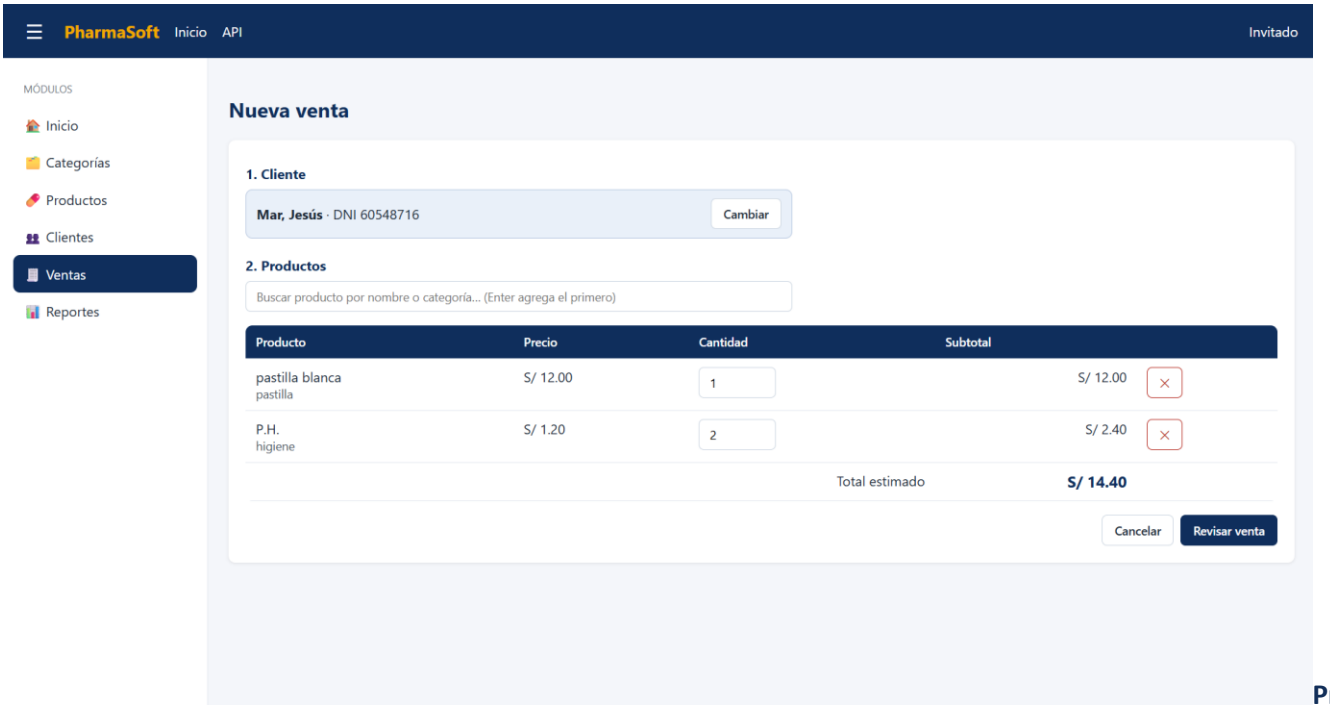


Prueba 05. Producto repetido

Resultado obtenido: El sistema permitió identificar correctamente que el producto «pastilla blanca» ya se encontraba agregado al detalle de la venta, mostrando su cantidad actual. Al seleccionar nuevamente la opción «+ Agregar», se incrementó automáticamente la cantidad de 1 a 2 unidades, sin generar una fila duplicada. Asimismo, el subtotal se actualizó de S/ 12.00 a S/

24.00 y el total estimado se recalculó correctamente, demostrando el funcionamiento de la validación de productos duplicados.

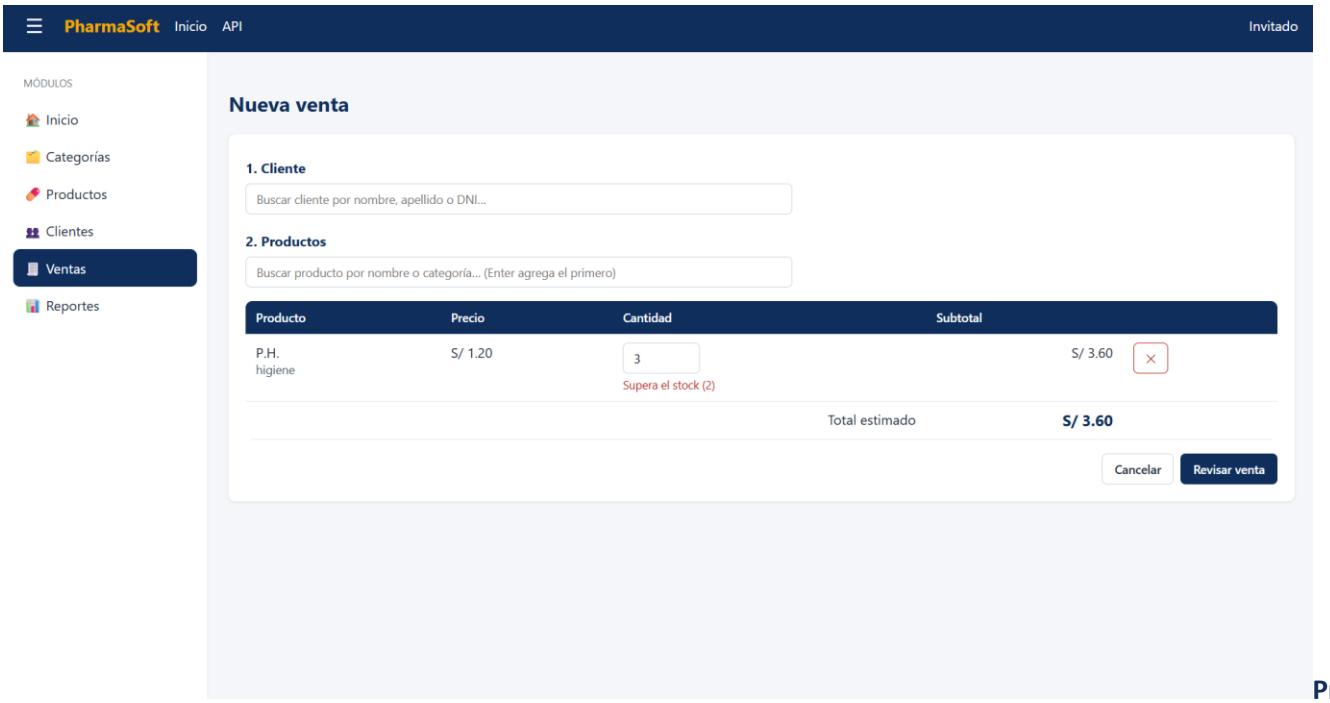




Prueba

06. Cantidad > stock

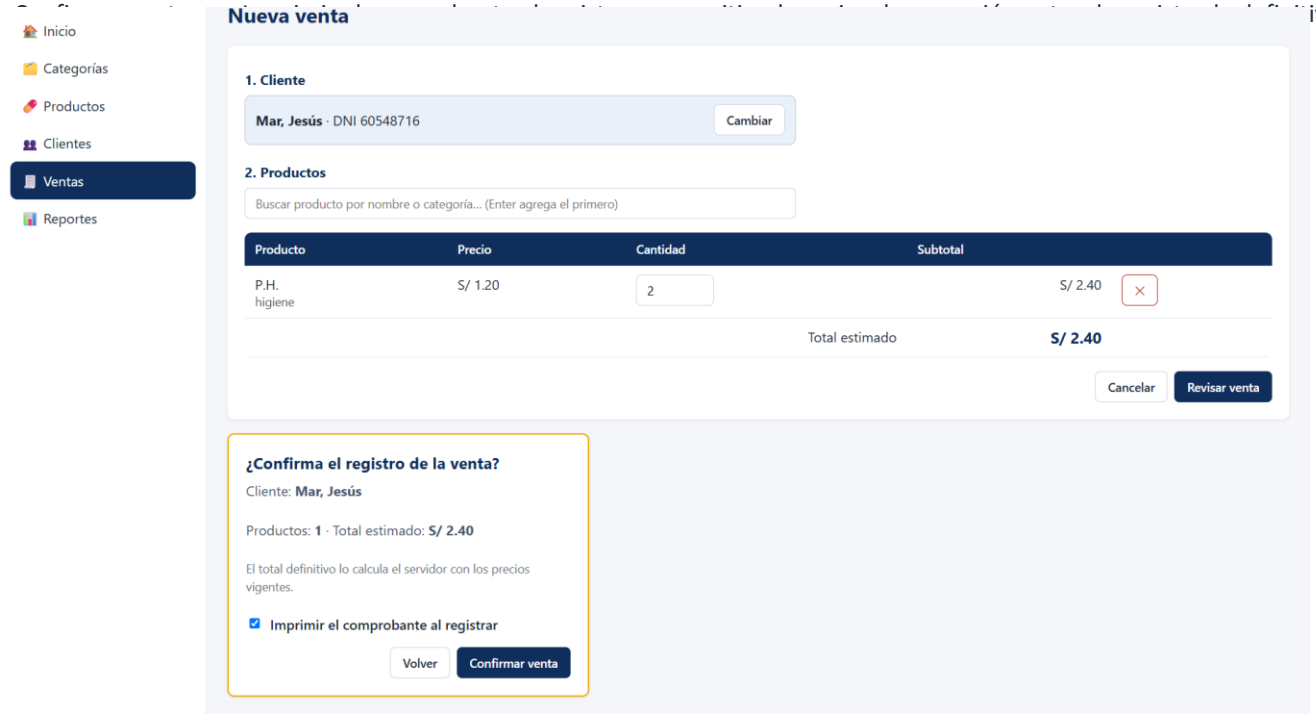
Resultado obtenido: Al ingresar una cantidad de productos superior al stock disponible, el sistema mostró el mensaje de advertencia «Supera el stock», indicando que la cantidad solicitada excedía las unidades disponibles. Asimismo, al presionar el botón «Revisar venta», el formulario impidió avanzar a la etapa de confirmación, evitando el registro de una venta con stock insuficiente.



Prueba

07. Revisar datos válidos

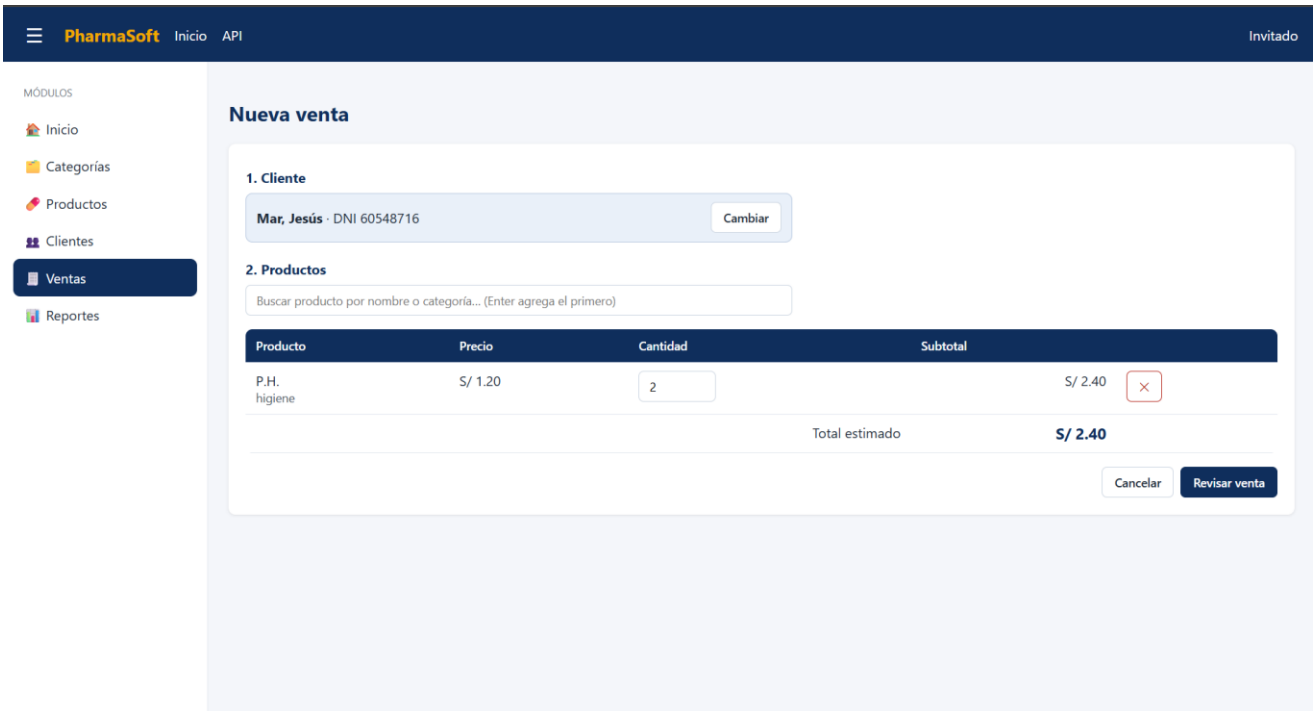
Resultado obtenido: El sistema mostró correctamente el panel «¿Confirma el registro de la venta?», presentando los datos del cliente, la cantidad de productos seleccionados y el total estimado de S/ 2.40. Asimismo, se visualizaron las opciones «Volver», [illegible] ivamente.

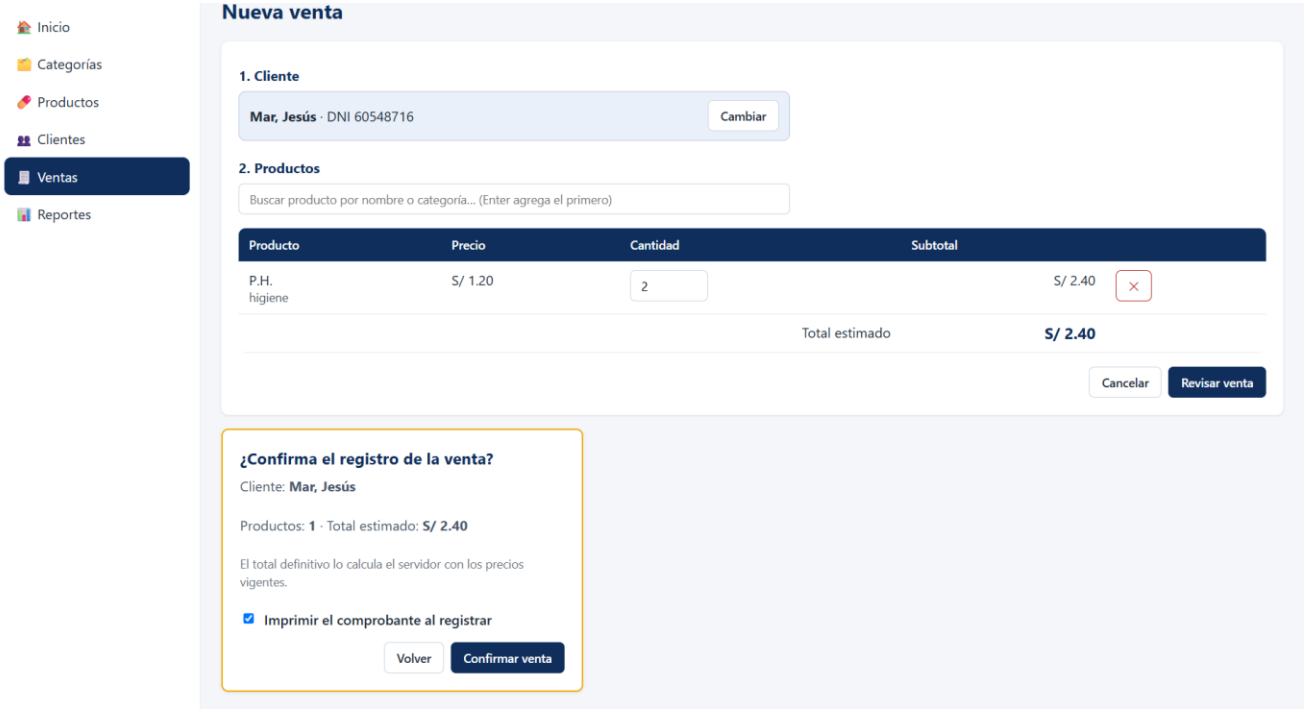


Prueba

08. Volver

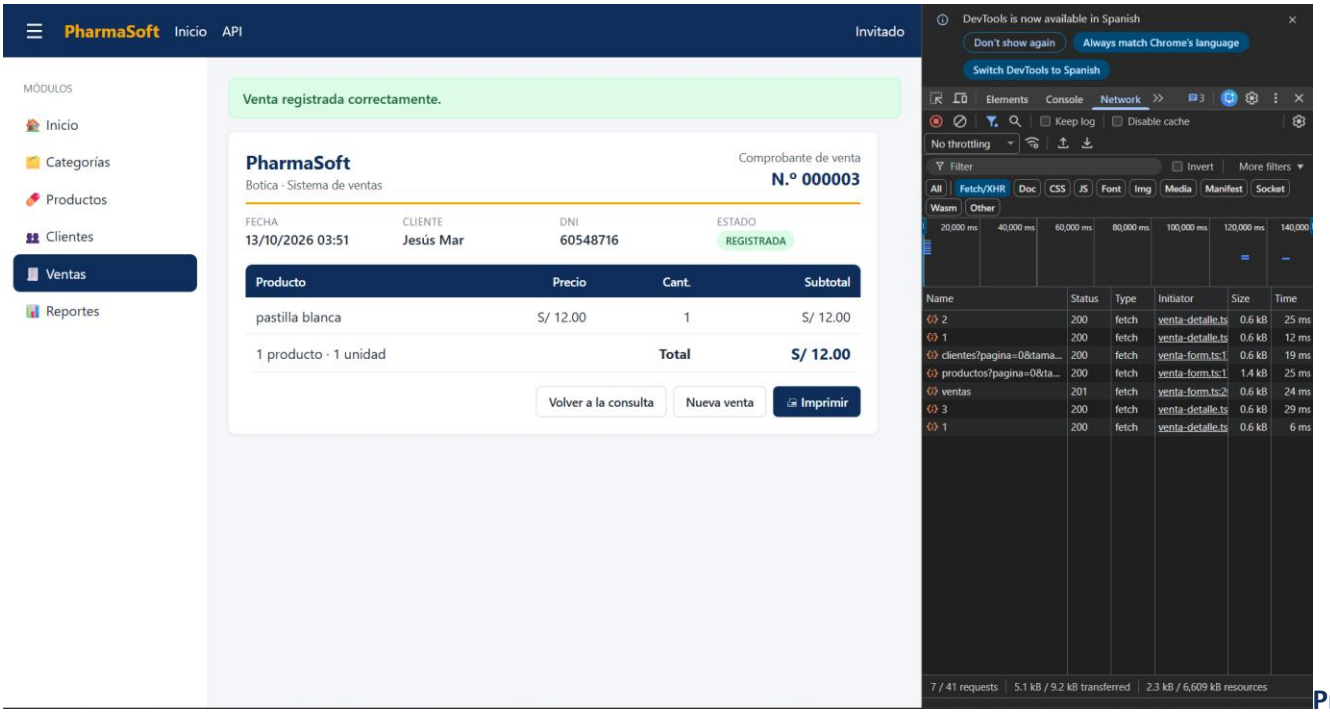
Resultado obtenido: El sistema cerró correctamente el panel de confirmación y regresó al formulario de Nueva venta, conservando el cliente seleccionado, el producto agregado, la cantidad de 2 unidades y el total estimado de S/ 2.40. No se produjo ninguna pérdida de información durante el retorno.





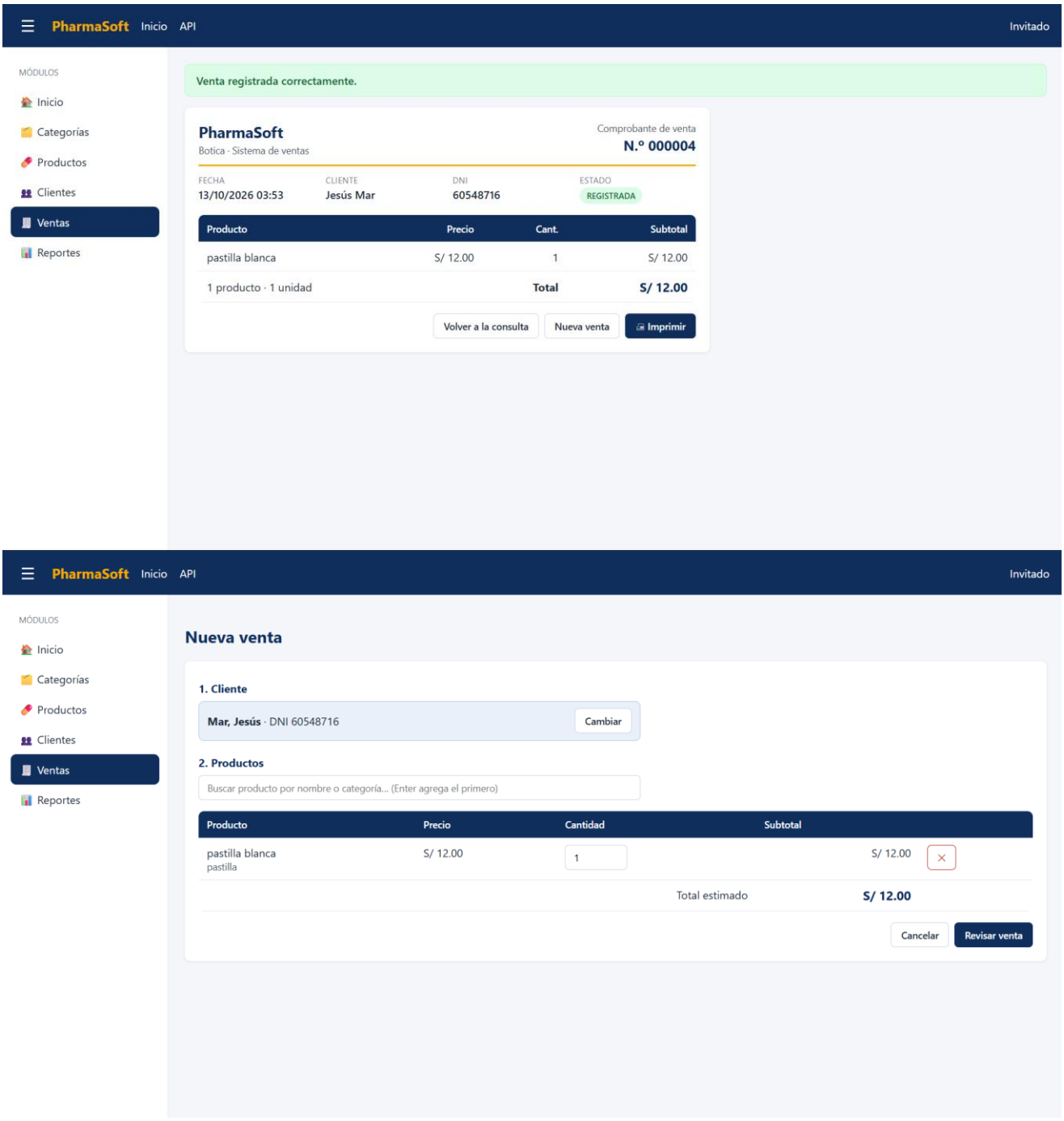
Prueba 09. Confirmar

Resultado obtenido: El sistema registró exitosamente la venta y mostró el mensaje «Venta registrada correctamente». En la pestaña Network se verificó que la petición POST al endpoint de ventas obtuvo el código HTTP 201 (Created). Asimismo, se generó el comprobante N.º 000003, correspondiente al cliente Jesús Mar, con el producto seleccionado, un total de S/ 12.00 y estado «REGISTRADA», confirmando la comunicación correcta entre el frontend Angular y PharmaBackend.



Prueba 10. Comparar totales

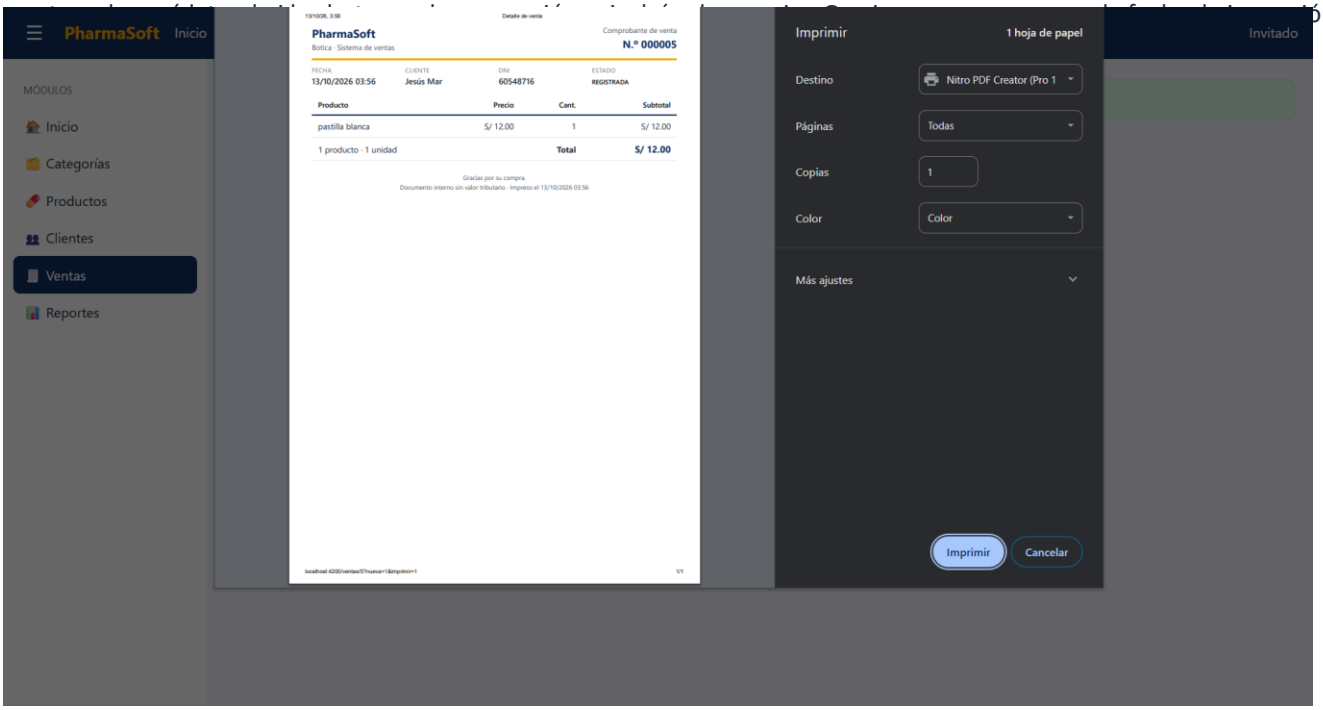
Resultado obtenido: El sistema generó correctamente el comprobante de venta N.º 000003, mostrando un total definitivo de S/ 12.00, calculado por PharmaBackend. Considerando el precio unitario de S/ 12.00 y la cantidad de 1 unidad, el cálculo corresponde al importe esperado en el formulario. Sin embargo, queda pendiente confirmar mediante una captura previa que el total estimado mostrado por Angular también fue de S/ 12.00 para esta misma operación.



Prueba

11. Impresión automática

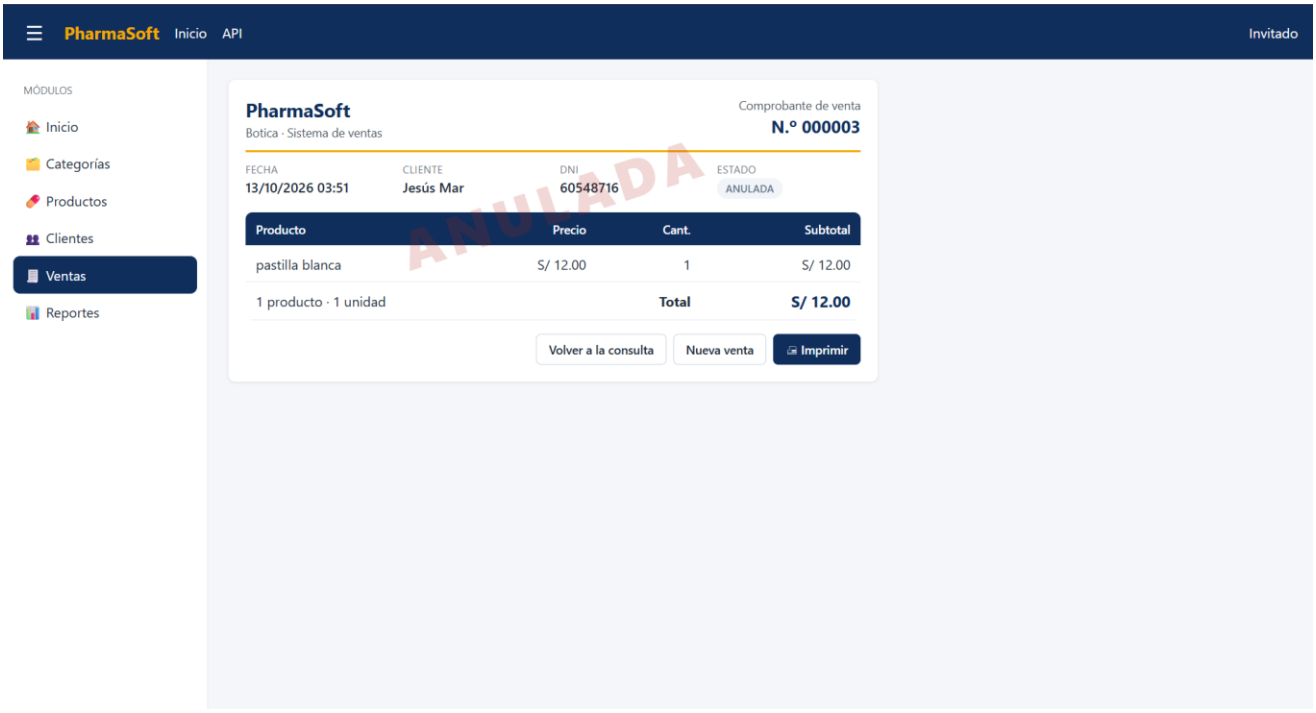
Resultado obtenido: El sistema abrió correctamente el diálogo de impresión del navegador, mostrando el comprobante N.º 000005 correspondiente al cliente Jesús Mar, identificado con DNI 60548716. La vista previa presentó el producto vendido, su cantidad, el precio unitario y el total definitivo de S/ 12.00. Asimismo, se comprobó que el documento estaba organizado en una sola página, sin



Prueba

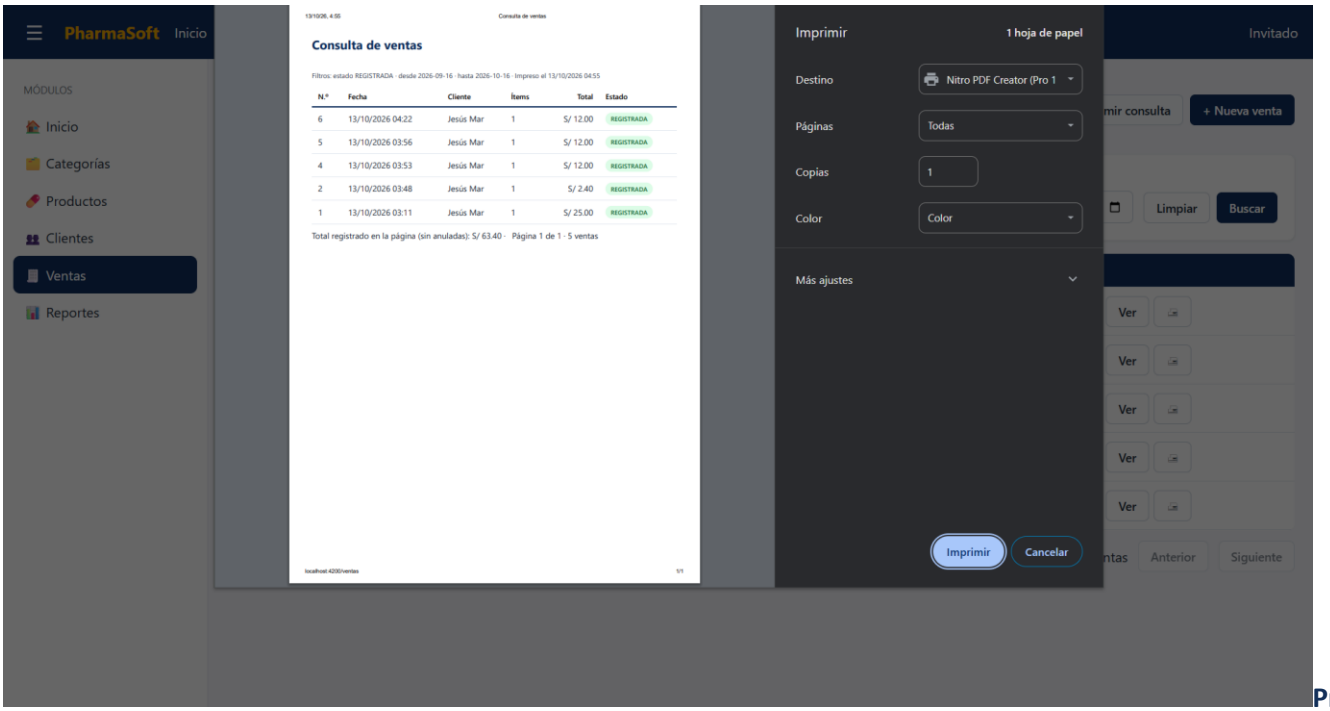
12. Comprobante anulado

Resultado obtenido**:** El sistema mostró correctamente el comprobante N.º 000003 correspondiente al cliente Jesús Mar, con estado «ANULADA» y un sello diagonal de identificación. Se conservaron los datos originales de la operación, incluyendo el producto «pastilla blanca», la cantidad de una unidad y el importe de S/ 12.00. Se comprobó satisfactoriamente la representación visual de una venta anulada.



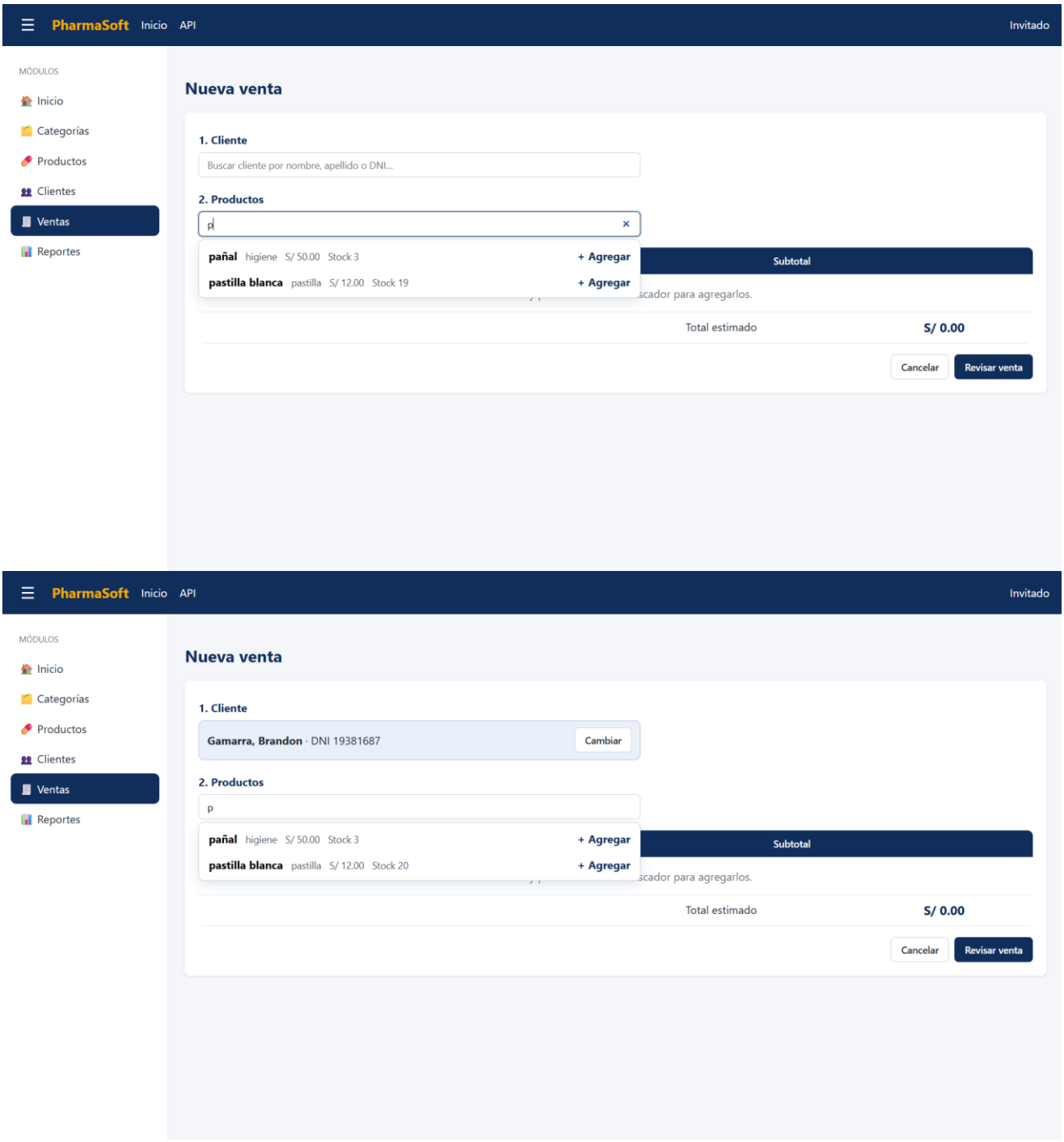
Prueba 13. Imprimir consulta

Resultado esperado: Tabla, filtros y total de página.



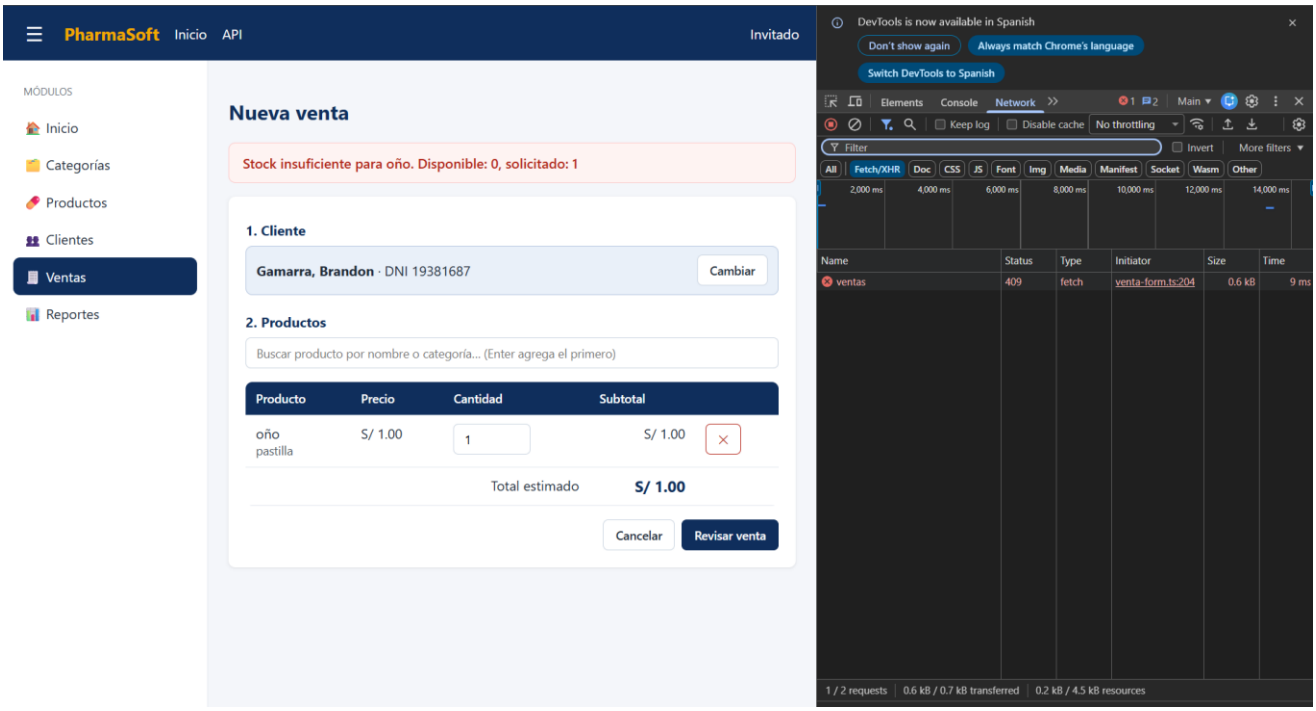
Prueba 14. Verificar stock

Resultado obtenido**:** El sistema permitió verificar el stock disponible de un producto antes y después de registrar una venta. Al realizar la operación, PharmaBackend procesó la cantidad vendida y actualizó el inventario, descontando las unidades correspondientes. De esta manera, se comprobó el funcionamiento de la actualización automática del stock y su integración con el módulo de ventas.



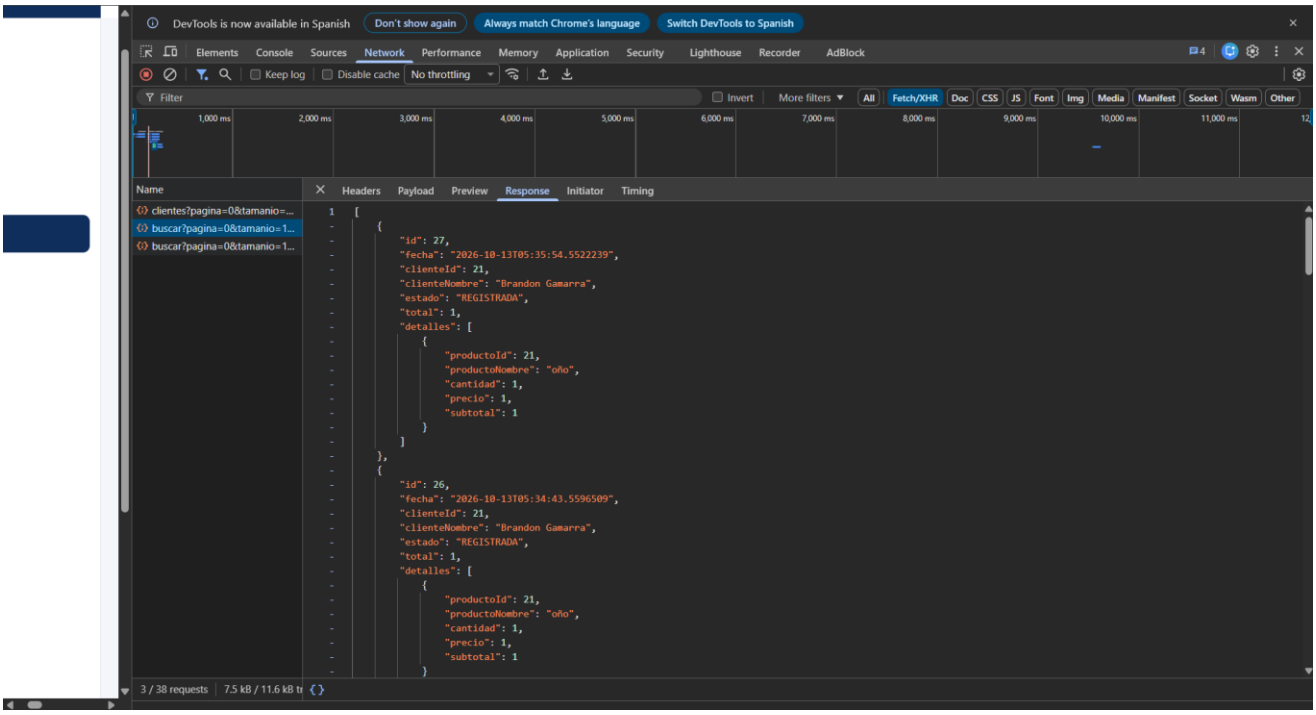
Prueba 15. Concurrencia con stock final

Resultado obtenido: **El sistema rechazó correctamente el registro de la venta y mostró el mensaje «Stock insuficiente para oño. Disponible: 0, solicitado: 1». Asimismo, se verificó que la petición POST al endpoint de ventas devolvió el código HTTP 409 (Conflict). La aplicación regresó al formulario de Nueva venta y conservó los datos ingresados, permitiendo corregir la operación. Esto evidencia que PharmaBackend vuelve a validar el stock antes de aceptar el registro.**



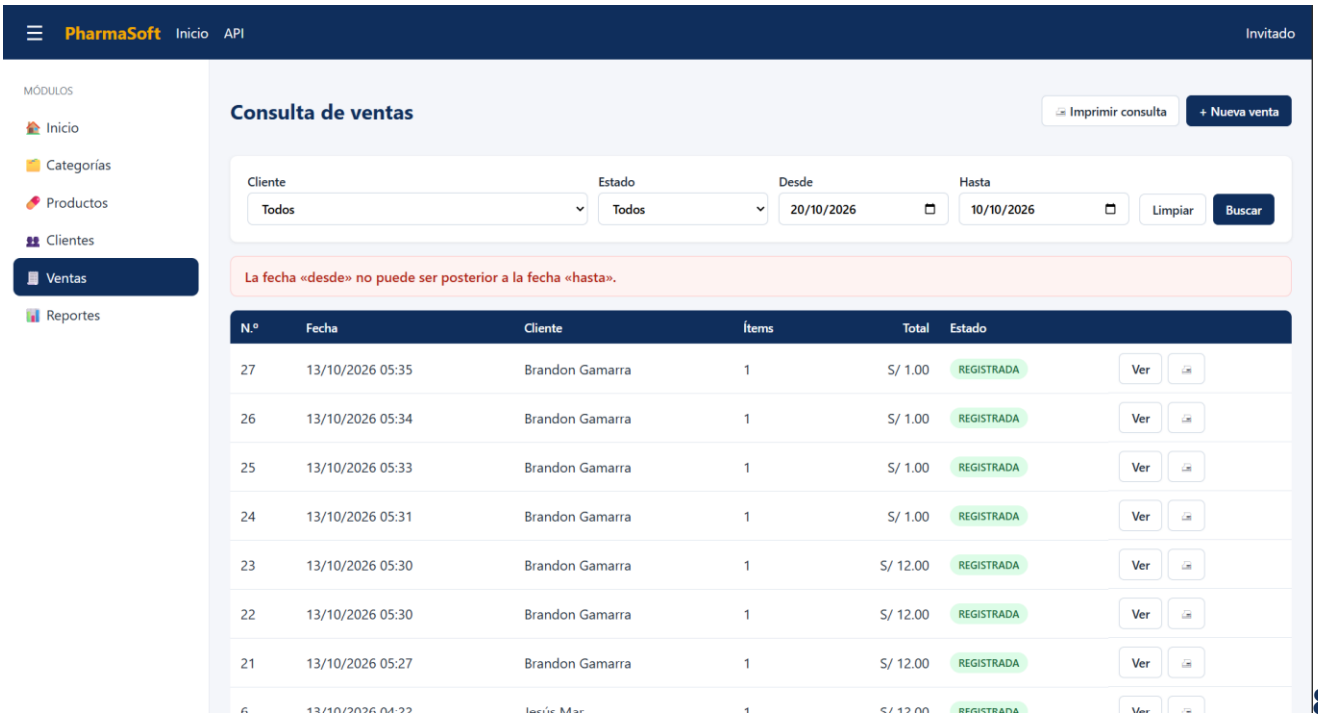
Prueba 16. Filtrar por cliente y fechas

Resultado obtenido: Se verificó mediante las herramientas de desarrollador del navegador que PharmaSoft realiza peticiones al endpoint de búsqueda de ventas, enviando correctamente los parámetros de paginación y ordenamiento (pagina=0, tamanio=10, ordenarPor=fecha y direccion=desc). Asimismo, se comprobó que no se incluyen parámetros de filtros cuando estos no tienen valores seleccionados.Estado:



Prueba 17. Rango de fechas invertido

Resultado obtenido: El sistema detectó correctamente que la fecha inicial era posterior a la fecha final y mostró el mensaje «La fecha “desde” no puede ser posterior a la fecha “hasta”». Asimismo, la aplicación permaneció en la pantalla de consulta de ventas sin presentar errores de funcionamiento, permitiendo al usuario corregir el rango ingresado.



8.

Evaluación de cumplimiento basada en código

Criterio	Resultado de revisión estática
Estructura de feature, modelos, rutas, servicio y tres páginas	Identificado en el Git Hub.
Buscadores, FormArray, signals y cálculos	Identificado en venta-form.ts.
Validación de cantidad, stock y confirmación	Identificado en venta-form.ts.
Servicio REST y filtros opcionales	Identificado en venta-service.ts.
Comprobante, impresión y consulta paginada	Identificado en componentes de detalle y lista.
Backend real, POST 201, rollback, impresión física	No comprobado con ejecución real.
Cuatro commits y solicitud de integración a develop	No comprobado: no se aportó historial Git remoto.